VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
VAISHNAVI MISHRA	24BCE10916
Slot	**Date of Submission**
B22	31 March 2026

1. Introduction

One big reason tech works the way it does now? Open source software. Many of the tools people rely on every day - apps, programs, systems - run because of openly shared code. Instead of locked-down licenses, users can change, copy, or pass along open-source code without barriers. Because of that freedom, others build on it, fix it, improve it together. My look into this world focuses on one example: VLC Media Player. That program became the case I examined. One thing about VLC stands out - it plays almost every kind of music and video file without fuss. Because it adapts well and rarely breaks down, people keep coming back to it. What drives this software becomes clearer when you look under the hood at its purpose and design choices. Built as open-source, anyone can inspect, tweak, or share improvements freely. That openness turns users into helpers, sometimes even coders who fix issues they notice. Running tasks from the terminal shows another layer of control beyond clicking icons. Five small programs were written just for testing core abilities on Linux machines. Gathering details about the machine comes first, then checking which apps are installed follows naturally. Storage space gets measured next, logs get scanned after that, each step building familiarity. Talking directly to the system like this reveals patterns hidden behind graphical screens. Even basic commands become powerful once combined in smart sequences. Freedom to study and change code makes learning feel less abstract than usual.

2. Chosen Software

Chosen for this task? The open-source pick lands on VLC Media Player. This tool pops up everywhere, built by folks at VideoLAN. Running smooth isn’t limited to one system - try it on Windows, hop over to Linux, swing by macOS, even tap into mobile setups

VLC Media Player works straight away, yet fits many needs easily. Most files run right inside it, since special tools aren’t required. Free to grab, built openly - people change it, pass it along just fine

VLC Media Player Features

One format plays here, another there - flexibility built in. Audio shifts smoothly between types, video follows close behind. Not every player handles both, yet this one does. Files arrive however they must, leaving room for change later on

• Free and open-source software

Running on different systems means flexibility across setups. Windows handles it just fine, yet Linux runs smoother in some cases. macOS fits into the mix without trouble. Each environment adapts differently depending on configuration details

• Simple and user-friendly ui

• Supports streaming

• Lightweight and fast performance

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

VLC Media Player began as a college experiment in France, built by learners at École Centrale Paris through the VideoLAN effort. Back then, few tools handled many file types smoothly - this one aimed to change that

Some clips came with sound. Others didn't have much beyond basic playback, which often meant fewer options. A few only handled single tasks, leaving people stuck.

A single issue stood out: people had to download separate codecs just to run various audio and video types. That turned things into a mess, slowed everything down. Besides, no open, cost-free option existed - nothing flexible enough to adjust on your own terms.

Starting with a fix for common playback problems, VLC took shape. Built to handle many file types right away - no extra software needed - it played videos smoothly. Streaming across networks? That mattered too, so it worked there as well. Ownership stayed loose; instead of locking things down, the team opened everything up. People everywhere got free access - not just to use but tweak and boost its power.

One choice pushed VLC toward becoming the top media player around. Millions now rely on it, thanks to how easily it adapts, stays clear to use, and remains open for anyone.

A2. What the license really means

Working on open source means folks join forces, swapping thoughts and skills so the outcome grows stronger. When people pool what they know, the result often beats any first try. Ideas bounce around, shaping something improved through shared effort. This way of building things lifts the whole project higher.

Some projects never appear online. A few exist only for company work. Firms spend cash and hours building tools. Profit often drives the choice to hold code back. Ownership stays private when returns matter.

Building products using open-source software? Companies do it all the time. As long as licenses are respected, most see it as fair play. Giving back matters just as much as taking. Think Red Hat tweaking tools or Google sharing code updates. Progress moves faster when fixes and upgrades flow both ways. The whole group gains when firms add their work instead of hoarding changes. Open exchange keeps things moving forward. Not every company joins in equally though. Some stick around without pitching in. Others dive deep, shaping projects from the start. What counts is playing by the rules while helping others along. Shared effort means stronger software for anyone who uses it.

When someone shares software freely, others can study how it works before making changes. Because of that, progress often moves faster than building alone. A shared effort tends to grow stronger when people add their own fixes. Starting fresh is possible but less common once a base exists. With access comes responsibility to contribute back thoughtfully.

Truth is, sharing code doesn't steal ideas - it grows them. One person's work becomes another's starting point, speeding things up. Problems get solved quicker when effort stacks, not repeats. Building begins where others left off, skipping old hurdles.

A3. The ethics of open source

Sharing code openly invites many hands, each adding small changes that grow into stronger results. One person starts, then another joins - improving, adjusting, building further. Ideas mix when people collaborate without hiding what they know. The outcome often turns out sharper, faster, or smarter than any single version could be alone. Progress

comes through constant tweaks made by those who care to contribute.

Some projects stay closed off. Not every piece opens up to the public. Business goals drive certain builds. Firms pour effort and cash into creation. That is why they hold back code. Profit motives shape those decisions. Keeping software private becomes a path forward.

Open source lets firms create tools using shared code - provided they stick to the rules and give something in return. Firms such as Red Hat or Google run on open tech while feeding upgrades into the mix, lifting others along the way

.

Truth is, open source doesn't steal creativity - it feeds teamwork, speeds up progress. With old answers already built, coders shift energy toward fresh puzzles. Starting there makes room for more invention, not less.

Part B — Linux Footprint (Unit 2)

One good thing about VLC Media Player is how it fits right into Linux setups. This tool runs smoothly on those operating systems. Installation feels familiar, much like other software there. File management follows what users already know.

Installation Method:
Take Ubuntu as an instance, running 'sudo install vlc' in the terminal does the job. Another path opens with Snap - type 'sudo snap install vlc' to grab the package that way. Either method gets the player onto your system without extra steps.

Key Directories:

Once installed, VLC Media Player will come in /usr/bin. Settings might also hide under /etc, depending on the system setup. Logs aren't something this app creates much of. Details on its behavior could turn up within /var/log if checked.

User and Group:
Alone in action, VLC Media Player runs like any standard program instead of hiding behind the scenes. This design means its permissions match those of the current user. System files remain untouched thanks to that boundary. Should a weakness emerge within the app, consequences cannot spread far.

Service Management:

Starting VLC Media Player typically does not happen behind the scenes like a web server would. Opening it often means typing vlc into the terminal yourself. Another way? Clicking on its icon through the desktop layout. When done, shutting it down comes from simply exiting the window. Or pressing Ctrl+C if still working inside the terminal space.

Update Model:

Someone's always tweaking VLC Media Player, fixing bits here and there. Out in the open, coders patch things up regularly. Through Linux package tools, those changes slide into your system. Type something like 'sudo update', then 'sudo apt upgrade' - that pulls fresh pieces. Keeps the player steady, shielded from nasties lurking around. Running those lines means it stays current without fuss.

Part C — The FOSS Ecosystem (Units 3 & 4)

One thing about VLC Media Player: it doesn't stand alone. Instead, because of its ties to shared code, it runs using several community-built parts. Thanks to those connections, it works smoothly while giving back - helping new tools grow alongside it.

Dependencies:

Starting off, VLC uses a range of open-source tools - FFmpeg handles how sound and moving images are unpacked. Its backbone comes from libVLC, forming the central structure around which everything else fits. Other format-specific decoder kits plug in to widen what kinds of files can play. Written mostly in C along with some C++, the code takes shape through standard Linux-based toolsets. Compilers and builders common to that system turn raw text into working software.

Impact and Contributions:

Starting strong with simplicity, VLC sparked ideas across countless apps focused on sound and video. Built right in, its core features found new life through tailored playback systems and live broadcasts. Sometimes hidden under the surface, libVLC hands coding power to creators shaping unique software experiences. That quiet engine inside keeps showing up where you least expect it.

Connection with Web Technologies and LAMP

VLC does not belong to the LAMP group - Linux, Apache, MySQL, PHP - but handles network media delivery well. While separate from core web server tools, it sends video and audio across connections. Web systems often pull streams from VLC just like files from a server. That link lets websites show live or recorded clips smoothly. So even though it stands apart, its role fits neatly into online media setups.

Community and Governance:

Out there among digital tools, VLC takes shape thanks to the VideoLAN group. Driven forward not just by core members but also a wide circle of contributors who care about open code. Talking happens across message boards, email threads, plus shared web spaces where ideas flow freely. People coding from different corners of the planet help patch flaws while slipping in fresh capabilities now and then. Oversight rests with the inner VideoLAN crew, keeping things transparent and rooted in collective effort.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative
Cost	Free	Comes with Windows OS (paid)
Security (who can audit the code?)	Source code is open and auditable	Source code is closed
Support and reliability	Community-based support	Community-based support
Freedom to modify	Users can modify the code	Modification not allowed
Community vs corporate control	Community-driven	Controlled by company
Your overall verdict	VLC Media Player is a better choice for most users because it is free, flexible, and supports a wide range of formats. It provides more control and transparency compared to proprietary software.	

What sets VLC apart is its openness - unlike closed programs such as Windows Media Player, anyone can tweak it. Because the code is free to access, people adjust features however they

want. Curiosity leads some to explore how things function under the surface. That kind of access builds clarity and control. Flexibility shows up not just in playback but in what you're allowed to do behind the scenes.

Still, big brands tend to back their tools with solid help teams plus smoother designs. Take Microsoft - spending piles on updates along with guidance makes things hold together well, especially if you are not into tech stuff.

Truth be told, VLC Media Player fits more people well since it handles many file types, costs nothing, while giving greater flexibility. Developers often lean on it when they need fine-grained control, so do others comfortable under the hood. For everyday tasks, it holds up strong - personally, I'd pick it without hesitation; down the line, helping out an open-source effort might just happen.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

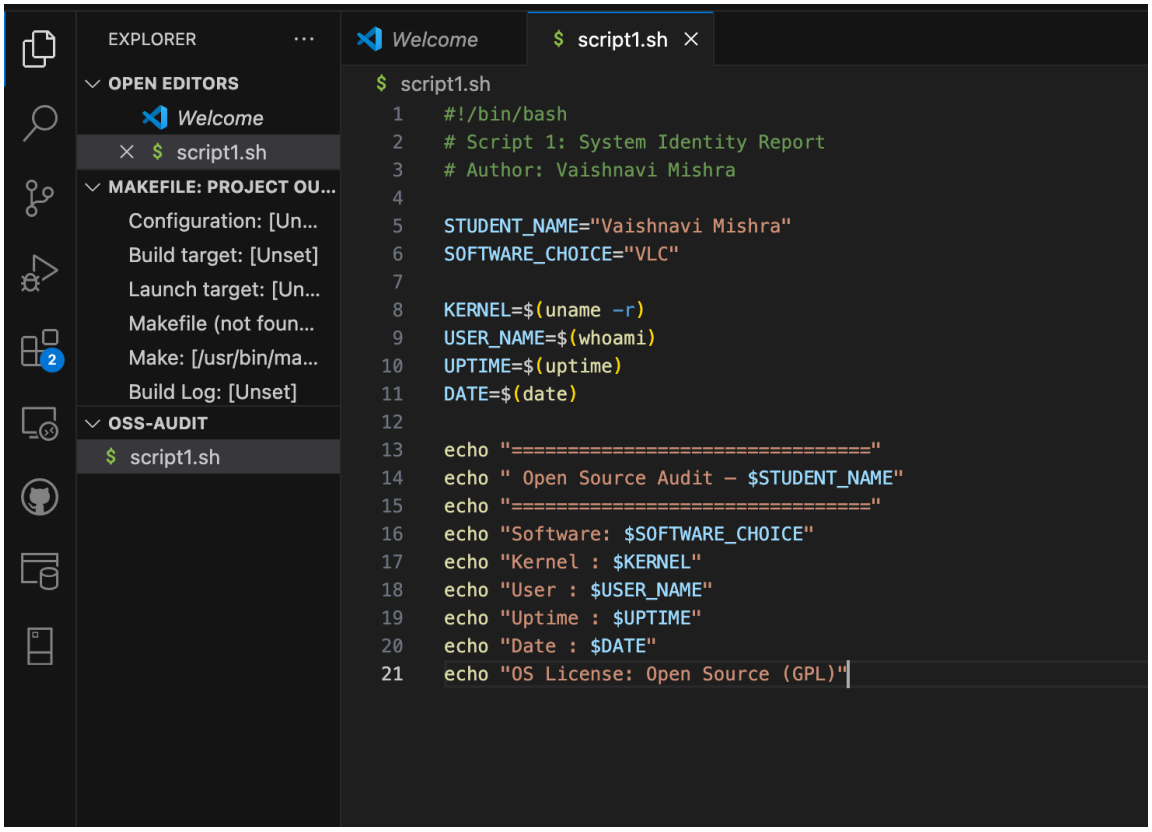
Out of nowhere comes a script tossing up facts about a Linux setup - laid out clean. Holding pieces like which kernel runs things, who just signed in, how long it has stayed awake, also what day and hour show now. Not loud. Just details, one after another, fitting together without fuss. Built to present what matters, nothing more, shaped so anyone can scan it fast. No extra noise, only what sits under the surface every time it runs.

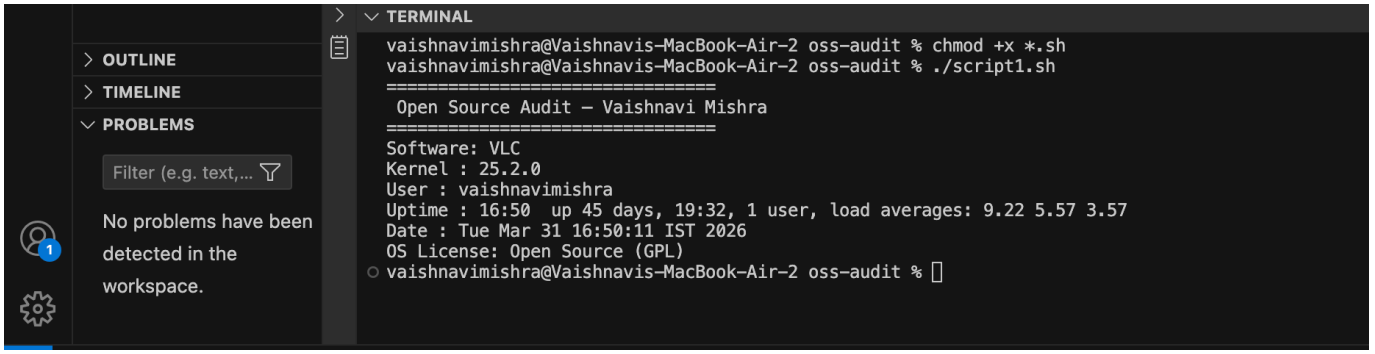
Information pulled by system commands gets saved using variables inside the script. Kernel version comes from running uname minus r. To find out who is currently logged in, the script runs whoami instead. Uptime reveals how many hours the machine has stayed active. What time it is right now appears when date executes.

A fresh start happens when the script pulls in custom details - like who the student is or what program they picked - to shape how things appear. Out pops the data, neatly arranged through styled messages that make the opening view clear, never messy. How it looks? Crisp lines meet focused layout, just enough to guide attention without clutter.
A notice pops up at the end, showing the OS runs on an open-source license like GPL. What you see here uses variables, slips in command results, then shapes how things appear on screen through basic shell scripting moves.

Stuff you can work with includes placeholders for values, showing stuff on screen, grabbing results from one bit of code to use in another, plus how things look when they print out.

Starter structure




OUTLINE
TIMELINE
PROBLEMS
Filter (e.g. text,...
No problems have been detected in the workspace.
TERMINAL
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit % chmod +x *.sh
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit % ./script1.sh
==========
Open Source Audit – Vaishnavi Mishra
==========
Software: VLC
Kernel : 25.2.0
User : vaishnavimishra
Uptime : 16:50 up 45 days, 19:32, 1 user, load averages: 9.22 5.57 3.57
Date : Tue Mar 31 16:50:11 IST 2026
OS License: Open Source (GPL)
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit %

Script 2 — FOSS Package Inspector (Unit 2)

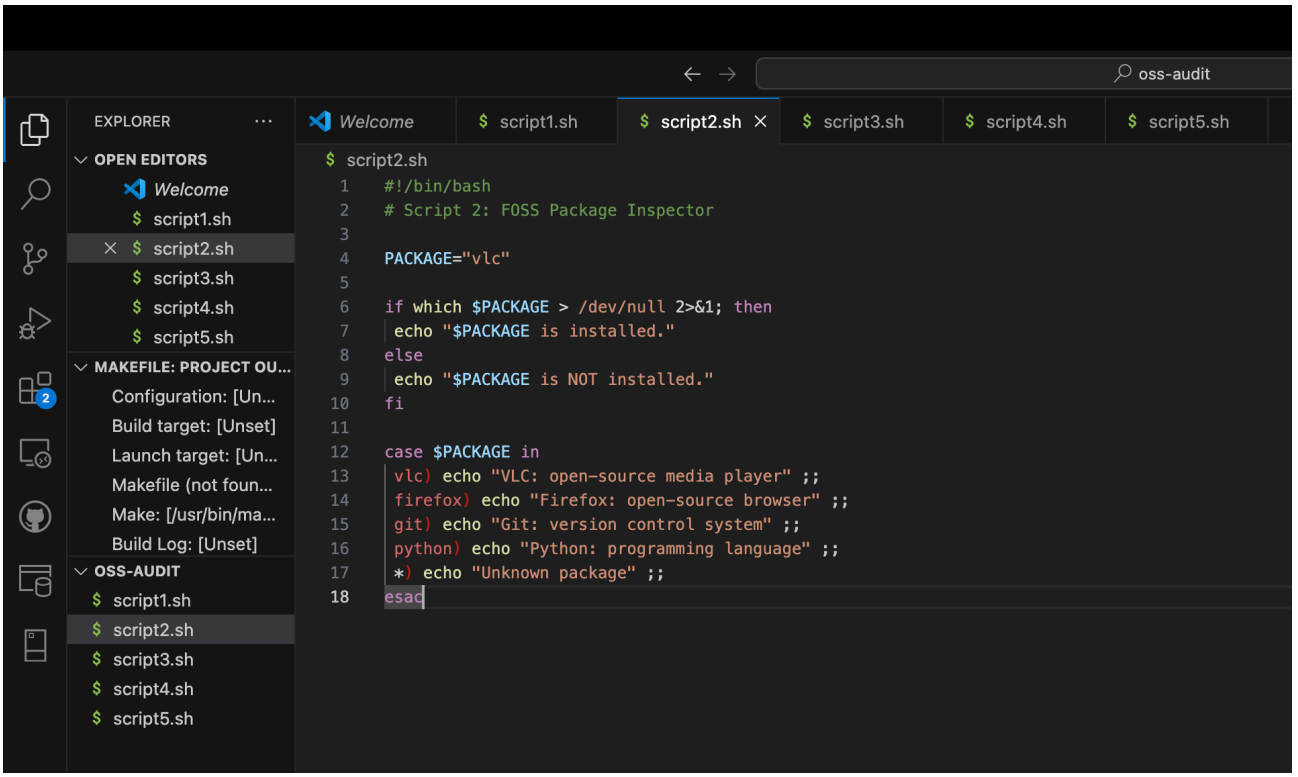
This time around, VLC serves as the test subject for spotting installed open-source tools. A look into the system path happens through the 'which' command. Availability of that program gets confirmed only when the path reveals it.

Should the system locate the package, a message confirms its presence. When nothing turns up, the output states it missing instead. Running without one reveals the absence clearly.

A quick explanation pops up depending on which tool you pick. When it's VLC, something shows. The same happens if Firefox comes up. Git triggers its own line too. Python has one as well. Anything else brings out a fallback note instead.

A closer look at how scripts check software begins with conditions that decide next steps. Depending on what the system reports, certain commands run instead of others. One path might examine installed tools while another skips ahead. Choices like these shape the flow based on real-time findings. Matching patterns often rely on structured branches called cases. Each option responds only when its label fits. Execution moves differently each time, guided by actual package details uncovered.

Starter structure




```
zsh: permission denied: ./script2.sh
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit % chmod +x script2.sh
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit % ./script2.sh
vlc is NOT installed.
VLC: open-source media player
```

Script 3 — Disk and Permission Auditor (Unit 2)

This script checks disk usage and access rights in key system folders. Inside it sits a collection called DIRS holding paths like /etc, /home, yet also /usr/bin.

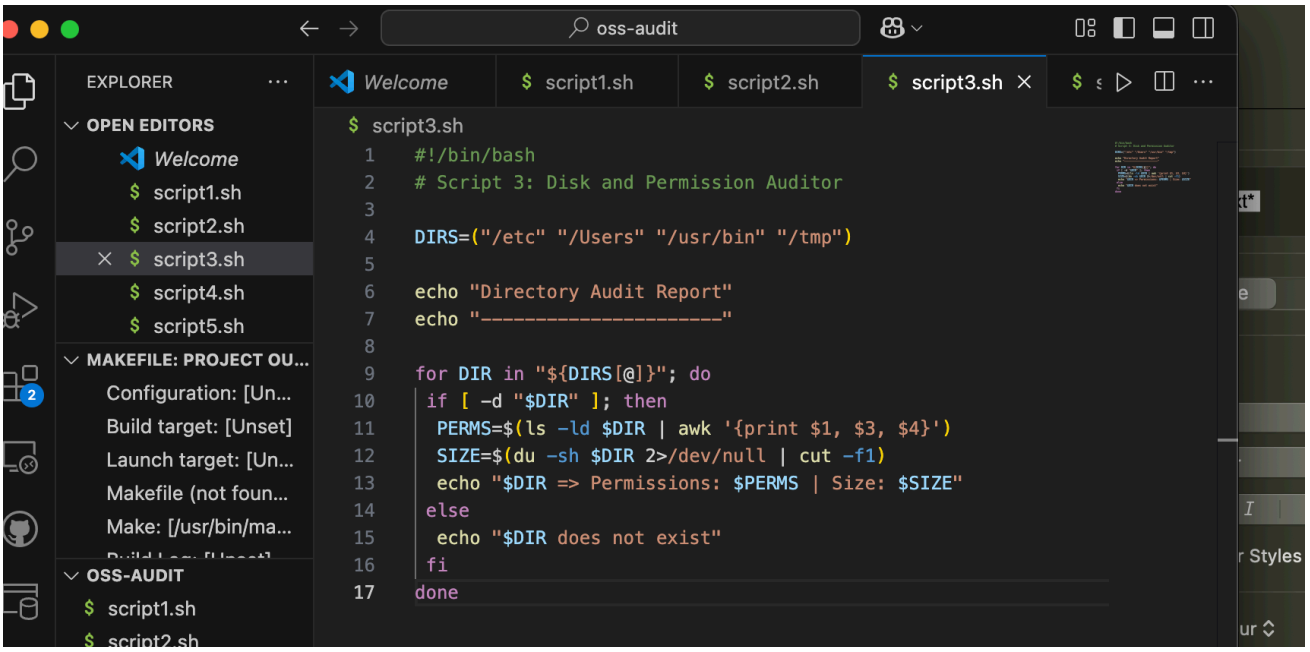
Looping helps go over each folder listed. When a folder comes up, existence gets checked by an if test. Should that folder be there, its name appears on screen before details follow. The 'ls -ld' command reveals access rights, user, and team tied to it right after.

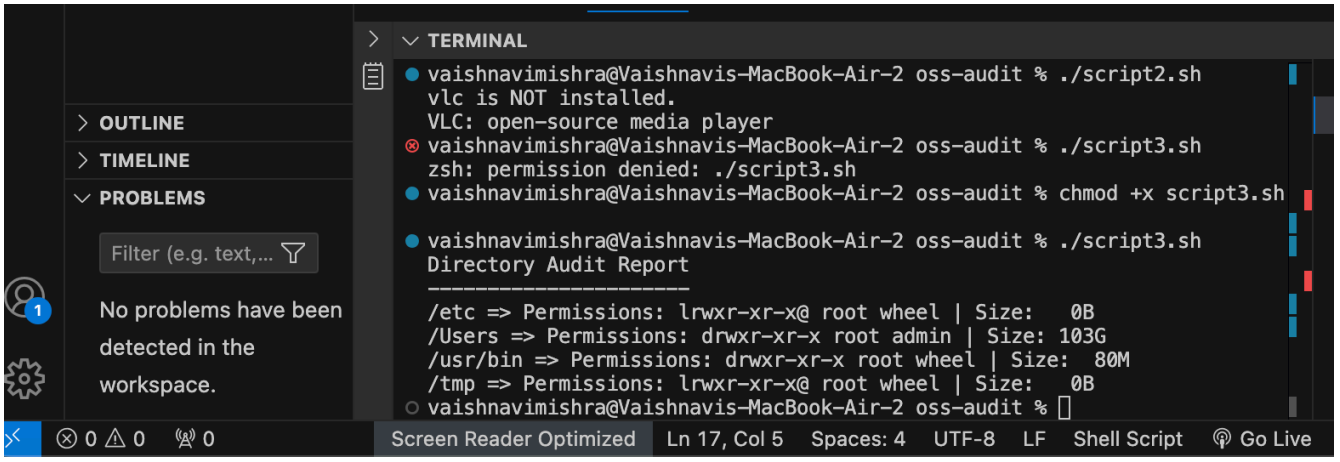
Starting off, the script figures out how much disk space every folder takes up through 'du -sh'. To keep things clean, any errors get sent into silence with '2>/dev/null', stopping clutter on screen.

A missing folder triggers the script to display a note about its absence. When there's no sign of the specified path, out pops a notice saying it is not located.

A loop steps through each item while checking size limits along the way. When thresholds trigger, actions shift based on what's found inside folders. Data collects into structured groups that track names and space used. Commands run outside the main flow pull live details from the machine itself. Decisions split paths if access rights block progress or allow it forward.

Starter structure





Script 4 — Log File Analyzer (Units 2 & 5)

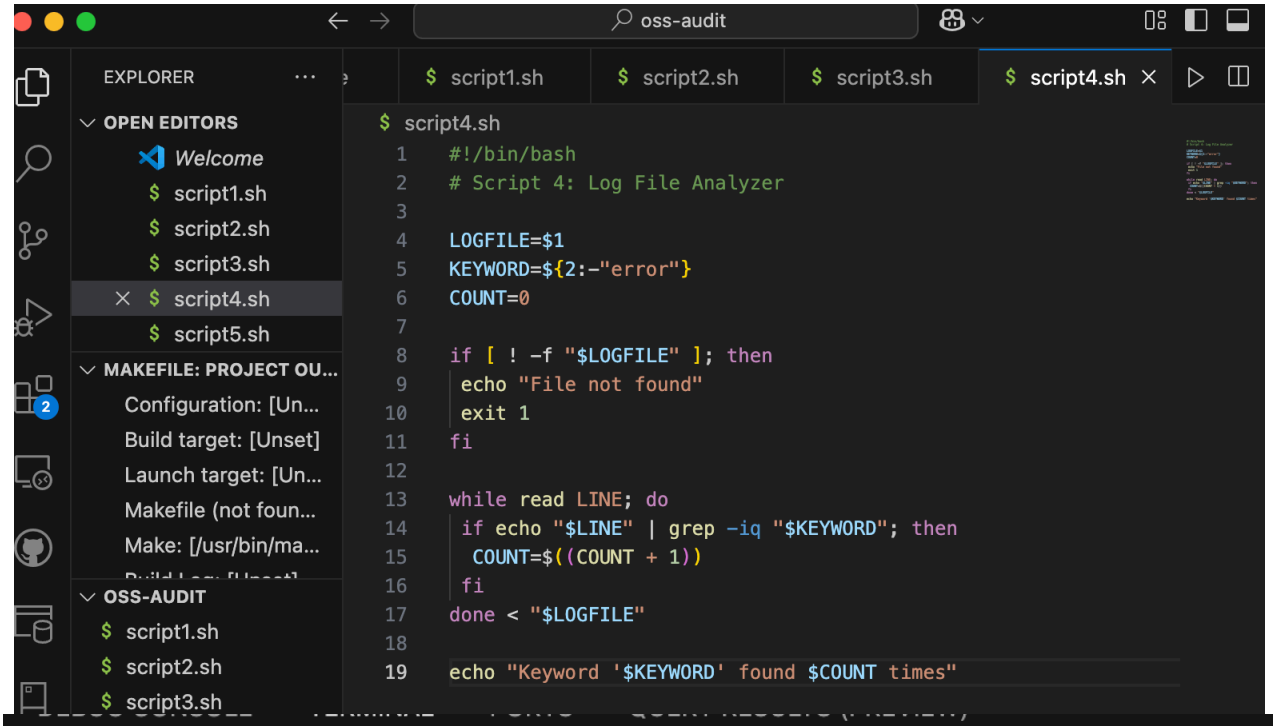
Line by line, this script checks a log file for certain words like error or warning. It takes the file name from what you type first when running it - that's the $1 part. When there's a second word supplied ($2), it searches for that instead. Default behavior kicks in if nothing follows - then it just looks for error. Counting matches happens quietly as it moves through each entry. No extra steps needed once started; everything runs straight through. The total number of matching lines shows up at the end. Behind the scenes, simplicity keeps things moving without hiccups. You get results based on exactly what was asked - no more, no less. Every run depends entirely on those two inputs given upfront.

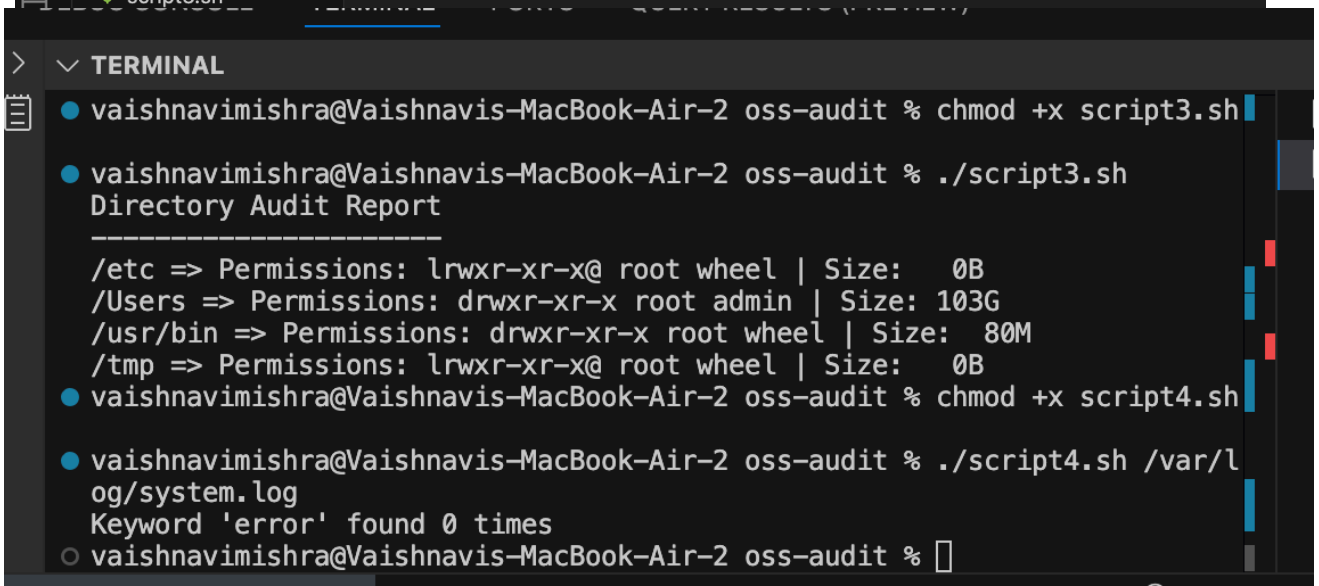
Out there, a check begins - does the file actually exist? When missing, up pops an error, then stops everything. Starting at zero, COUNT gets set to count how many lines match later on.

A single pass through the lines happens using a while-read setup. Each time, before moving on, the system tests the present line against the target word without caring about uppercase or lowercase letters. When there's a hit, instead of skipping ahead, the tally goes up by one.

Once every line is handled, out comes a tally - how often the word showed up. What you see here uses loops, decisions based on conditions, inputs from the terminal, along with reading files, all inside shell scripts.

Starter structure





Script 5 — The Open Source Manifesto Generator (Unit 5)

This script begins with a message shown on screen. Once the person answers, it moves forward. A series of prompts appear one after another. One question explores which tools feel right in their hands. Freedom gets described not as a word but as something lived. Another prompt digs into what kind of thing they wish to make. Input shapes everything that follows. The output becomes a short declaration shared freely. Each answer changes how it looks. Nothing stays fixed once words are given.

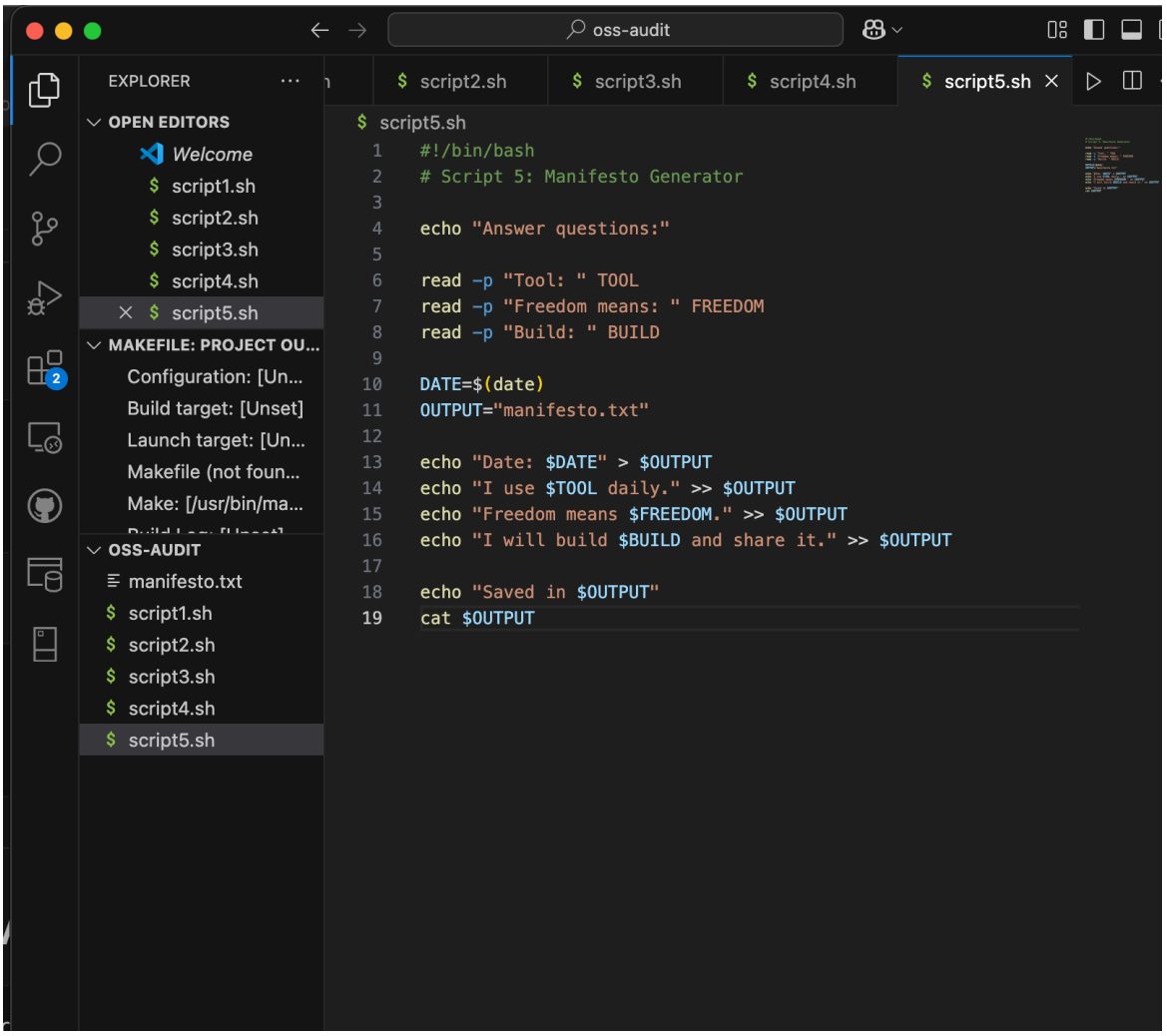
A moment passes before the script tucks away what you typed into labels like TOOL, FREEDOM, and BUILD. After that, a call to 'date' pulls today's system date - slipping it quietly into DATE.

A new file called 'manifesto.txt' appears when you type '>' first. After that, extra lines slip in each time '>>' runs. Sentences form from what users typed, fitting together like puzzle pieces. Into the file they go, one after another, shaped by input.

A prompt appears once everything finishes, letting you know it worked. After that, what's inside the new file pops up right there using cat.

Starting off, the script shows how users can enter data during runtime. Moving ahead, stored values help manage that information efficiently. Sometimes, words get pieced together using format tricks you might not expect. Dates pop up on screen thanks to a built-in system tool. Files come into play when saving or grabbing saved content. Finally, results appear clearly so anyone can see what happened.

Starter structure



```
Keyword "error" found 0 times
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit % chmod +x script5.sh

./script5.sh
Answer questions:
Tool: VLC
Freedom means: sharing
Build: Free app
Saved in manifesto.txt
Date: Tue Mar 31 17:07:55 IST 2026
I use VLC daily.
Freedom means sharing.
I will build Free app and share it.
vaishnavimishra@Vaishnavis-MacBook-Air-2 oss-audit %
```

Screen Reader Optimized | Ln 19, Col 12 | Spaces: 4 | UTF-8 | LF | Shell Script | Go Live

A closing thought

Looking back, working on this made clear why open-source matters now more than ever. VLC Media Player turned out to be more than something you play videos with - it showed how people building together create real value. When code is shared, one person's work becomes a stepping stone for others, leading to smarter fixes and stronger tools over time.

Working on this gave me hands-on practice with Linux along with script writing for the terminal. Little programs turned out to handle routine jobs automatically, which made workflows smoother. In the end, doing this deepened my grasp of open-source - not just how it works but why it matters.